Fiber Architecture for Group Extensions in Lean 4:

Cocycles, Splitting, and the Cohomological Bridge

Nova Spivack

2026

Abstract

We present a machine-checked formalization of the section 2-cocycle, splitting criterion, and embedding-problem equivalence for group extensions in Lean 4 [1], built on Mathlib's `GroupExtension` API [2, 3]. The formalization is organized around a *fiber architecture* that decomposes extensions into layers: fibers over quotient elements (N-torsors), set-theoretic sections (always exist), and homomorphic splittings (may not exist). The central results are: (1) the *splitting criterion*—an extension splits if and only if some section has trivial cocycle; (2) the *embedding-problem equivalence*—an abstract embedding problem (the central lifting problem in inverse Galois theory) is solvable if and only if the associated extension splits, if and only if the cocycle obstruction vanishes; (3) the *2-cocycle identity* connecting our construction to Mathlib's $\texttt{IsMulCocycle}_2$ predicate; and (4) a *local-to-global obstruction principle* via restriction to subgroups. The work fills documented gaps in Mathlib's `GroupExtension` module and takes a first step toward the cohomological classification of extensions. Companion work applies the same fiber architecture to Quillen's Theorem A for Galois connections. All results are fully machine-checked with zero `sorry`.

Keywords: Lean 4, Mathlib, group extensions, 2-cocycle, splitting criterion, group cohomology, interactive theorem proving, formal verification

MSC 2020: 20E22, 20J06, 68V15, 68V20

Code/Data availability: All Lean source files are available in the supplementary material under `InfinityCompression/GeneralMethod/GroupExtension/` and `InfinityCompression/GeneralMethod/Descent/`. The repository will be made publicly available upon publication.

Contents

1 Introduction **3**

2 Background **4**
- 2.1 Group extensions in algebra . . . 4
- 2.2 Mathlib's GroupExtension API . . . 5
- 2.3 Group cohomology in Lean . . . 5

3 Fiber Architecture **6**
- 3.1 Extension fibers . . . 6
- 3.2 Kernel action and torsor structure . . . 6
- 3.3 Set-theoretic sections vs. homomorphic splittings . . . 7

4 The 2-Cocycle and Splitting Criterion **7**
4.1 The section defect . . . 7
4.2 The 2-cocycle . . . 8
4.3 The defect formula . . . 8
4.4 The splitting criterion . . . 8
4.5 Section difference and independence . . . 9

5 Conjugation Action and 2-Cocycle Identity **10**
5.1 The conjugation action via sections . . . 10
5.2 Commutativity simplifications . . . 10
5.3 Splitting gives a group homomorphism . . . 11
5.4 The 2-cocycle identity . . . 11

6 Restriction to Subgroups and Local-to-Global **12**
6.1 The direct product extension . . . 12
6.2 The obstruction theorem . . . 12
6.3 Restriction via pullback subgroups . . . 13
6.4 Global-to-local and the contrapositive . . . 13

7 Embedding Problems as Extension Splitting **14**

8 Faithfully Flat Descent as Conservative Instance **14**
8.1 The descent principle . . . 15
8.2 Architectural contrast . . . 15

9 Related Work **15**

10 Conclusion **16**

A Theorem Index **17**

B Module Dependency Graph **18**

1 Introduction

Group extensions—short exact sequences $1 \to N \to E \xrightarrow{\pi} G \to 1$—are a classical organizing tool in algebra. The question of when such a sequence *splits*, i.e., when π admits a group-homomorphic right inverse, is central to the structure theory of groups and connects directly to group cohomology: the obstruction to splitting is a class in $H^2(G, N)$.

Despite the maturity of the underlying mathematics, the formalization landscape is incomplete. Mathlib's `GroupExtension` module [2, 3] provides the basic definitions—`GroupExtension`, `Section`, `Splitting`, the conjugation action `conjAct`, and equivalences of extensions—but its documentation contains explicit TODOs for the cohomological classification:

> *"If N is abelian, [...] there is a bijection between equivalence classes of group extensions and `groupCohomology.H2` (which is also stated as a TODO in `GroupCohomology/LowDegree.lean`)."*
> —Mathlib, `GroupExtension/Defs.lean`, lines 43–52

This paper takes a first step toward filling that gap by formalizing the section 2-cocycle, proving the splitting criterion, and establishing the 2-cocycle identity that connects our construction to Mathlib's $\texttt{IsMulCocycle}_2$ predicate.

What is new. The underlying mathematics—2-cocycles, the splitting criterion, coboundaries, restriction to subgroups—is classical and well-known (see, e.g., Brown [8], Robinson [10]). Our contribution is the *formalization* of this material in Lean 4 and its *organization* into a layered architecture that exposes the intermediate invariants (fibers, sections, cocycles) as first-class proof objects. To our knowledge, no prior formalization in any proof assistant provides the section cocycle and splitting criterion at this level of generality, connected to a library-standard cohomological predicate.

The fiber architecture. Our approach is organized around a *fiber decomposition* of the extension. For each $g \in G$, the fiber $\pi^{-1}(g)$ is an N-torsor: the kernel $N \cong \ker \pi$ acts freely and transitively. A set-theoretic section $\sigma : G \to E$ with $\pi \circ \sigma = \mathrm{id}_G$ always exists (since π is surjective), but a *homomorphic* section may not. The defect is measured by the 2-cocycle $\mathfrak{c}_\sigma(g_1, g_2) = \iota^{-1}\big(\sigma(g_1)\,\sigma(g_2)\,\sigma(g_1 g_2)^{-1}\big)$, which takes values in N. The extension splits if and only if some section has identically trivial cocycle.

This decomposition into layers—fibers, sections, cocycles, splittings—provides a structured interface for proof engineering. Rather than treating splitting as a monolithic predicate, the architecture exposes the intermediate invariants that classical algebra uses to analyze extensions.

Contributions.

1. **Fiber architecture** (§3): We define `ExtensionFiber`, prove nonemptiness (surjectivity), and establish the N-torsor structure via free and transitive kernel action.

2. **Section 2-cocycle and splitting criterion** (§4): We define `sectionCocycle`, prove the defect formula `section_mul_eq`, and establish the biconditional `splits_iff_trivial_cocycle`.

3. **Coboundary and section-independence** (§4.5): We define `sectionDiff` (the N-valued difference between two sections) and prove that the splitting question is independent of the section choice.

4. **Conjugation action and 2-cocycle identity** (§5): We connect the section cocycle to Mathlib's `IsMulCocycle₂` via the conjugation action, proving the 2-cocycle identity in full.

5. **Restriction to subgroups and local-to-global** (§6): We formalize `restrictExtension` via pullback subgroups and prove `global_split_implies_local_split` with its contrapositive.

6. **Embedding problems as extension splitting** (§7): We formalize abstract embedding problems and prove `solvable_iff_splits` and `solvable_iff_trivial_cocycle`, connecting the central lifting problem of inverse Galois theory to cocycle obstructions.

As an illustrative comparison, §8 briefly formalizes faithfully flat descent—a *conservative* instance of the same architectural pattern where the obstruction vanishes—using existing Mathlib results.

Place in a broader series. This paper isolates the cohomological group-extension and embedding-problem component of a broader fiber/section/obstruction architecture. Companion work [12] applies the same architecture to Quillen's Theorem A for Galois connections, demonstrating that the method reaches beyond extension theory into combinatorial topology.

Proof status. The formalization comprises twelve Lean modules totaling approximately 1500 lines of code: seven core group extension modules, two supplementary modules (`NonAbelianObstruction.lean`, `SchurMultiplier.lean`), an adapter module (`MathlibAdapter.lean`), an embedding-problem module (`EmbeddingProblem.lean`), and a module for faithfully flat descent. All results compile against Mathlib v4.29.0-rc6 with zero `sorry`.

Outline. Section 2 reviews group extensions and Mathlib's API. Section 3 presents the fiber architecture. Section 4 develops the 2-cocycle, splitting criterion, and section-independence. Section 5 connects the cocycle to group cohomology via the conjugation action. Section 6 formalizes restriction to subgroups. Section 7 formalizes embedding problems as extension splitting. Section 8 treats faithfully flat descent as an illustrative comparison. Section 9 surveys related work, and Section 10 concludes.

2 Background

2.1 Group extensions in algebra

A *group extension* of G by N is a short exact sequence of groups

$$1 \longrightarrow N \xrightarrow{\iota} E \xrightarrow{\pi} G \longrightarrow 1, \tag{1}$$

where ι is injective, π is surjective, and $\mathrm{im}(\iota) = \ker(\pi)$. The group E is called the *middle term* or *extension*. The extension *splits* if there exists a group homomorphism $s : G \to E$ with $\pi \circ s = \mathrm{id}_G$; equivalently, $E \cong N \rtimes G$ for some action of G on N.

Given any set-theoretic section $\sigma : G \to E$ (i.e., a function with $\pi \circ \sigma = \mathrm{id}_G$, which exists since π is surjective), one defines the *2-cocycle*

$$\mathfrak{c}_\sigma(g_1, g_2) \;=\; \iota^{-1}\big(\sigma(g_1)\,\sigma(g_2)\,\sigma(g_1 g_2)^{-1}\big) \;\in\; N. \tag{2}$$

The product $\sigma(g_1)\,\sigma(g_2)\,\sigma(g_1 g_2)^{-1}$ lies in $\ker(\pi) = \mathrm{im}(\iota)$ because π is a homomorphism and σ is a section; injectivity of ι ensures the preimage is unique.

The 2-cocycle satisfies the *cocycle identity*

$$\mathfrak{c}_\sigma(g_1, g_2) \cdot \mathfrak{c}_\sigma(g_1 g_2, g_3) \;=\; \varphi_\sigma(g_1)\big(\mathfrak{c}_\sigma(g_2, g_3)\big) \cdot \mathfrak{c}_\sigma(g_1, g_2 g_3), \tag{3}$$

where $\varphi_\sigma(g)(n) = \iota^{-1}\big(\sigma(g)\,\iota(n)\,\sigma(g)^{-1}\big)$ is the conjugation action of G on N through the section. When N is abelian and the action is written additively, this is the standard condition defining a 2-cocycle in $Z^2(G, N)$.

Changing the section σ to σ' modifies the cocycle by a *coboundary*: if $d(g) = \iota^{-1}\big(\sigma(g)\,\sigma'(g)^{-1}\big)$, then $\mathfrak{c}_{\sigma'}$ and $\mathfrak{c}_\sigma$ differ by a coboundary involving d. The splitting question—whether $\mathfrak{c}_\sigma$ is trivial for some σ—depends only on the cohomology class $[\mathfrak{c}_\sigma] \in H^2(G, N)$.

2.2 Mathlib's GroupExtension API

Mathlib's formalization, due to Yamazaki [3], provides the following structures in `Mathlib.GroupTheory.GroupExten`

- `GroupExtension N E G`: a structure bundling a group homomorphism $\iota : N \to^* E$ (called `inl`), a group homomorphism $\pi : E \to^* G$ (called `rightHom`), injectivity of ι, the range-kernel equation $\mathrm{im}(\iota) = \ker(\pi)$, and surjectivity of π. Throughout this paper, we write `S.inl` for ι and `S.rightHom` for π, following Mathlib's naming.
- `GroupExtension.Section`: a function $\sigma : G \to E$ with $\pi \circ \sigma = \mathrm{id}_G$ (set-theoretic right inverse), accessed via `S.Section`.
- `GroupExtension.Splitting`: a section that is also a group homomorphism, bundling a `MonoidHom G E` with a proof that it is a right inverse of `rightHom`. Accessed via `S.Splitting`; the projection `toSection` recovers the underlying set-theoretic section.
- `GroupExtension.conjAct`: the conjugation action $E \to^* \mathrm{Aut}(N)$, defined via `MulAut.conjNormal`. Concretely, `conjAct`$(e)(n) = \iota^{-1}(e \cdot \iota(n) \cdot e^{-1})$; the key specification lemma is `inl_conjAct_comm`: $\iota(\texttt{conjAct}(e)(n)) = e \cdot \iota(n) \cdot e^{-1}$.
- `GroupExtension.Equiv`: equivalence of extensions via commuting diagrams.
- `GroupExtension.IsConj`: N-conjugacy of splittings.
- `SemidirectProduct.toGroupExtension`: the canonical extension $1 \to N \to N \rtimes_\varphi G \to G \to 1$.

Mathlib also provides `surjInvRightHom`, a canonical set-theoretic section constructed via `Function.surjInv` (the noncomputable right inverse of a surjective function), and `Section.mul_inv_mem_range_inl` which states that for any two sections σ, σ', the element $\sigma(g) \cdot \sigma'(g)^{-1}$ lies in $\mathrm{im}(\iota)$.

2.3 Group cohomology in Lean

Mathlib's `GroupCohomology/LowDegree.lean` defines:

- `IsMulCocycle₂ f`: the predicate that $f : G \times G \to M$ satisfies $f(gh, j) \cdot f(g, h) = g \bullet f(h, j) \cdot f(g, hj)$ for all g, h, j.
- `IsMulCoboundary₂ f`: the predicate that f is a coboundary, i.e., there exists $x : G \to M$ with $g \bullet x(h)/x(gh) \cdot x(g) = f(g, h)$.

Our 2-cocycle identity (§5.4) shows that `sectionCocycle` satisfies `IsMulCocycle₂` with respect to the conjugation action, thereby connecting the extension-theoretic cocycle to Mathlib's cohomological framework.

3 Fiber Architecture

The fiber architecture decomposes a group extension S : `GroupExtension` N E G into layers organized around the fibers of the quotient map $\pi = S.$`rightHom`.

3.1 Extension fibers

Definition 3.1 (Extension fiber)**.** For $g : G$, the *extension fiber* over g is the preimage $\pi^{-1}(g)$, formalized as a subtype:

```
def ExtensionFiber (g : G) : Type u :=
  { e : E // S.rightHom e = g }
```

Theorem 3.2 (Fiber nonemptiness)**.** *Every fiber is nonempty:*

```
theorem extension_fiber_nonempty (g : G) :
    Nonempty (ExtensionFiber S g)
```

Proof. Immediate from the surjectivity of $\pi = S.$`rightHom`: given $g : G$, surjectivity yields $e : E$ with $\pi(e) = g$, and $\langle e, \text{proof}\rangle$ inhabits the fiber. □

The surjectivity of π also yields a global set-theoretic right inverse:

Theorem 3.3 (Set-theoretic section existence)**.** *The quotient map admits a right inverse:*

```
theorem rightHom_surjective_hasRightInverse :
    Function.HasRightInverse S.rightHom
```

Mathlib provides a canonical such section via `Function.surjInv`:

```
noncomputable def canonicalFiberWitness (g : G) :
    ExtensionFiber S g :=
  ⟨Function.surjInv S.rightHom_surjective g,
   Function.surjInv_eq S.rightHom_surjective g⟩
```

3.2 Kernel action and torsor structure

The kernel $\ker(\pi) = \mathrm{im}(\iota)$ acts on each fiber by left multiplication through ι:

Definition 3.4 (Kernel action)**.**
```
def kernelActOnFiber (g : G) (n : N)
    (x : ExtensionFiber S g) : ExtensionFiber S g :=
  ⟨S.inl n * x.val, by simp [S.rightHom_inl, x.property]⟩
```

The well-definedness of this action—that $\iota(n) \cdot x$ still lies in the fiber over g—follows from $\pi(\iota(n)) = 1$ (the range-kernel equation) and $\pi(x) = g$.

Theorem 3.5 (Free action)**.** *The kernel action is free: if $\iota(n) \cdot x = x$ for some x in the fiber, then $n = 1$.*

```
theorem kernelAct_free (g : G) (n : N)
    (x : ExtensionFiber S g)
    (h : kernelActOnFiber S g n x = x) : n = 1
```

Proof. From $\iota(n) \cdot x = x$ we obtain $\iota(n) = 1_E$ by right-cancellation in E. Injectivity of ι then gives $n = 1$. □

Theorem 3.6 (Transitive action)**.** *The kernel action is transitive: for any two elements x, y in the same fiber, there exists $n : N$ with $\iota(n) \cdot x = y$.*

```
theorem kernelAct_transitive (g : G)
    (x y : ExtensionFiber S g) :
    exists n : N, kernelActOnFiber S g n x = y
```

Proof. The element $y \cdot x^{-1}$ lies in $\ker(\pi)$ (since $\pi(y) = \pi(x) = g$), hence in $\mathrm{im}(\iota)$ by the range-kernel equation. Choosing a preimage n with $\iota(n) = y \cdot x^{-1}$ gives $\iota(n) \cdot x = y$. □

Corollary 3.7 (N-torsor structure)**.** *Each fiber $\pi^{-1}(g)$ is an N-torsor: the kernel acts freely and transitively. In particular, all fibers have the same cardinality as N.*

This torsor structure is the fiber-level content of the first isomorphism theorem for groups: the fibers of a surjective homomorphism are the cosets of the kernel, and the kernel acts on its own cosets by translation.

3.3 Set-theoretic sections vs. homomorphic splittings

The fiber architecture makes visible a fundamental distinction:

Theorem 3.8 (Section existence)**.** *Every group extension admits a set-theoretic section:*

```
theorem section_exists : Nonempty S.Section :=
  ⟨S.surjInvRightHom⟩
```

Theorem 3.9 (Splitting characterization)**.** *The extension admits a splitting if and only if some section is a group homomorphism:*

```
theorem splitting_iff_section_is_homomorphism :
    Nonempty S.Splitting <->
    exists sgm : S.Section,
      forall g1 g2 : G,
        sgm.toFun (g1 * g2) =
          sgm.toFun g1 * sgm.toFun g2
```

Proof. The forward direction extracts the underlying section from a splitting and uses `map_mul'`. The reverse direction constructs a `Splitting` from a multiplicative section, deriving `map_one'` from the multiplicativity condition applied to $\sigma(1 \cdot 1) = \sigma(1) \cdot \sigma(1)$. □

The gap between Theorems 3.8 and 3.9 is the mathematical content of extension theory: set-theoretic sections always exist, but homomorphic ones may not. The 2-cocycle (§4) measures this gap.

4 The 2-Cocycle and Splitting Criterion

4.1 The section defect

Given a set-theoretic section $\sigma : S.$`Section`, the product $\sigma(g_1) \cdot \sigma(g_2) \cdot \sigma(g_1 g_2)^{-1}$ measures the failure of σ to be multiplicative. This element lies in $\ker(\pi) = \mathrm{im}(\iota)$:

```
private theorem section_defect_mem_range
    (sgm : S.Section) (g1 g2 : G) :
    sgm.toFun g1 * sgm.toFun g2 *
      (sgm.toFun (g1 * g2))^(-1) ∈ S.inl.range
```

[1]

The proof applies π to the product, uses the section law $\pi \circ \sigma = \mathrm{id}_G$ and the homomorphism property of π, and obtains $g_1 \cdot g_2 \cdot (g_1 g_2)^{-1} = 1$.

4.2 The 2-cocycle

Definition 4.1 (Section 2-cocycle)**.** The *2-cocycle* of a section σ is the N-valued function obtained by pulling back the defect through ι:

```
noncomputable def sectionCocycle
    (sgm : S.Section) (g1 g2 : G) : N :=
  (section_defect_mem_range S sgm g1 g2).choose
```

Theorem 4.2 (Cocycle specification)**.** *The cocycle satisfies the defining equation:*

```
theorem sectionCocycle_spec (sgm : S.Section)
    (g1 g2 : G) :
    S.inl (sectionCocycle S sgm g1 g2) =
      sgm.toFun g1 * sgm.toFun g2 *
        (sgm.toFun (g1 * g2))^(-1)
```

This is the Lean rendering of equation (2). The use of `Exists.choose` / `Exists.choose_spec` extracts the witness from the membership proof.

4.3 The defect formula

Rearranging the cocycle specification yields a multiplicative decomposition:

Theorem 4.3 (Defect formula)**.**
```
theorem section_mul_eq (sgm : S.Section)
    (g1 g2 : G) :
    sgm.toFun g1 * sgm.toFun g2 =
      S.inl (sectionCocycle S sgm g1 g2) *
        sgm.toFun (g1 * g2)
```

This says that the product of section values at g_1 and g_2 equals the "correction term" $\iota(\mathfrak{c}(g_1, g_2))$ times the section value at $g_1 g_2$. When the cocycle is trivial, σ is multiplicative.

4.4 The splitting criterion

Theorem 4.4 (Splitting criterion)**.** *An extension splits if and only if some section has trivial cocycle:*

```
theorem splits_iff_trivial_cocycle :
    Nonempty S.Splitting <->
    exists sgm : S.Section,
      forall g1 g2 : G,
        sectionCocycle S sgm g1 g2 = 1
```

[1] Lean 4 source uses Unicode symbols ($\in$, $\neq$, $\langle\rangle$); we render these as LaTeX math in listings throughout.

Proof. Forward direction. Given a splitting s, its underlying section s.`toSection` has trivial cocycle: by `sectionCocycle_spec`, $\iota(\mathfrak{c}(g_1, g_2)) = s(g_1) \cdot s(g_2) \cdot s(g_1 g_2)^{-1}$, which equals 1 by s.`map_mul'`. Injectivity of ι gives $\mathfrak{c}(g_1, g_2) = 1$.

Reverse direction. Given a section σ with trivial cocycle, the defect formula (Theorem 4.3) with $\mathfrak{c} \equiv 1$ gives $\sigma(g_1) \cdot \sigma(g_2) = \sigma(g_1 g_2)$ (after $\iota(1) = 1$ and simplification). We construct a `Splitting` with `toFun` $= \sigma$, deriving `map_one'` from $\sigma(1) \cdot \sigma(1) = \sigma(1)$ (the multiplicativity applied to $g_1 = g_2 = 1$) and `map_mul'` directly from the trivial-cocycle hypothesis. □

This theorem is the central result of the formalization. It provides the foundational layer toward Mathlib's TODO for the cohomological classification: the section cocycle and the criterion for splitting are the first step toward the full H^2 bijection.

4.5 Section difference and independence

Two sections σ, σ' of the same extension assign elements to the same fiber at each $g \in G$. Their difference lands in the kernel.

Definition 4.5 (Section difference)**.**
```
noncomputable def sectionDiff
    (sgm sgm' : S.Section) (g : G) : N :=
  (section_diff_mem_range S sgm sgm' g).choose
```

with specification:

```
theorem sectionDiff_spec (sgm sgm' : S.Section)
    (g : G) :
    S.inl (sectionDiff S sgm sgm' g) =
      sgm.toFun g * (sgm'.toFun g)^(-1)
```

The membership proof `section_diff_mem_range` verifies that $\sigma(g) \cdot \sigma'(g)^{-1} \in \ker(\pi)$ by applying π and using both section laws.

Theorem 4.6 (Section relation)**.** *Any section can be expressed in terms of another via the difference function:*

```
theorem section_eq_inl_diff_mul
    (sgm sgm' : S.Section) (g : G) :
    sgm.toFun g =
      S.inl (sectionDiff S sgm sgm' g) *
        sgm'.toFun g
```

This is the fiber-level statement that $\sigma(g)$ and $\sigma'(g)$ lie in the same N-torsor, differing by a kernel element.

The splitting criterion (Theorem 4.4) quantifies existentially over sections. The following corollaries record the explicit interface.

Theorem 4.7 (Section-independence)**.** *The splitting question depends only on the existence of* some *section with trivial cocycle, not on a specific choice:*

```
theorem splitting_independent_of_section :
    (exists sgm : S.Section,
      forall g1 g2 : G,
        sectionCocycle S sgm g1 g2 = 1)
    <-> Nonempty S.Splitting
```

Theorem 4.8 (Splitting from any trivial cocycle)**.** *If* any *specific section has trivial cocycle, the extension splits:*

```
theorem splits_of_any_trivial_cocycle (sgm : S.Section)
    (h : forall g1 g2 : G,
      sectionCocycle S sgm g1 g2 = 1) :
    Nonempty S.Splitting
```

In classical group cohomology, section-independence is a consequence of the coboundary formula: changing the section changes the cocycle by a coboundary, and a cocycle is trivial if and only if it is a coboundary of the zero function. Our formalization captures the conclusion (independence) directly from the biconditional structure of `splits_iff_trivial_cocycle`, without requiring the full coboundary calculus.

5 Conjugation Action and 2-Cocycle Identity

5.1 The conjugation action via sections

Mathlib provides `S.conjAct : E →* MulAut N`, the conjugation action of E on N through the embedding ι. Composing with a section gives an action of G on N:

Definition 5.1 (Section conjugation action)**.**
```
noncomputable def sectionConjAct
    (sgm : S.Section) (g : G) : MulAut N :=
  S.conjAct (sgm.toFun g)
```

Concretely, $\varphi_\sigma(g)(n) = \iota^{-1}\big(\sigma(g) \cdot \iota(n) \cdot \sigma(g)^{-1}\big)$. The specification theorem records this:

Theorem 5.2 (Conjugation specification)**.**
```
theorem inl_sectionConjAct (sgm : S.Section)
    (g : G) (n : N) :
    S.inl (sectionConjAct S sgm g n) =
      sgm.toFun g * S.inl n * (sgm.toFun g)^(-1)
```

5.2 Commutativity simplifications

When N is abelian (`CommGroup N`), conjugation by kernel elements is trivial:

Theorem 5.3 (Trivial kernel conjugation)**.** *For commutative N:*

```
theorem conjAct_inl (n m : N') :
    S'.conjAct (S'.inl n) m = m
```

Proof. We have $\iota(n)\cdot\iota(m) = \iota(m)\cdot\iota(n)$ by commutativity of N (applied through ι), so the conjugation $\iota(n) \cdot \iota(m) \cdot \iota(n)^{-1} = \iota(m)$, and injectivity of ι gives the result. □

Theorem 5.4 (Section-independence of conjugation)**.** *For commutative N, the conjugation action is independent of the section choice:*

```
theorem sectionConjAct_independent
    (sgm sgm' : S'.Section) (g : G') (n : N') :
    sectionConjAct S' sgm g n =
      sectionConjAct S' sgm' g n
```

Proof. By `Section.mul_inv_mem_range_inl`, there exists $m : N$ with $\iota(m) = \sigma(g) \cdot \sigma'(g)^{-1}$. Then $\sigma(g) = \iota(m) \cdot \sigma'(g)$, and the conjugation through $\sigma(g)$ factors as conjugation through $\iota(m)$ followed by conjugation through $\sigma'(g)$. By Theorem 5.3, the $\iota(m)$-conjugation is trivial, leaving only the $\sigma'(g)$-conjugation. □

This result is significant for the cohomological interpretation: when N is abelian, the G-action on N is well-defined (independent of the section), which is a prerequisite for defining $H^2(G, N)$.

5.3 Splitting gives a group homomorphism

When the section is a splitting (group homomorphism), the conjugation action is itself a group homomorphism:

```
noncomputable def splittingConjAct (s : S.Splitting) :
    G ->* MulAut N :=
  S.conjAct.comp s.toMonoidHom
```

5.4 The 2-cocycle identity

The central connection to group cohomology is the 2-cocycle identity:

Theorem 5.5 (2-cocycle identity)**.** *For any section σ:*

```
theorem sectionCocycle_isMulCocycle2_conj
    (sgm : S.Section) (g1 g2 g3 : G) :
    sectionCocycle S sgm g1 g2 *
      sectionCocycle S sgm (g1 * g2) g3 =
    sectionConjAct S sgm g1
      (sectionCocycle S sgm g2 g3) *
      sectionCocycle S sgm g1 (g2 * g3)
```

This is the Lean rendering of equation (3). It states that `sectionCocycle` satisfies the multiplicative 2-cocycle condition ($\texttt{IsMulCocycle}_2$) with respect to the conjugation action φ_σ.

Proof strategy. The proof proceeds by injectivity of ι: both sides, when mapped through ι, reduce to $\sigma(g_1) \cdot \sigma(g_2) \cdot \sigma(g_3) \cdot \sigma(g_1 g_2 g_3)^{-1}$ after expanding the cocycle specifications and performing associativity and cancellation in E. The key step is to right-multiply both sides by $\sigma(g_1 g_2 g_3)$ and then cancel via `section_mul_eq`, which collapses the intermediate section values. The formalization unfolds `sectionCocycle_spec` for all four cocycle terms and `inl_sectionConjAct` for the conjugation, then discharges the remaining equalities by associativity and cancellation in E.

Supporting lemmas that are fully proved include the triple product expansions:

```
theorem section_triple_product (sgm : S.Section)
    (g1 g2 g3 : G) :
    sgm.toFun g1 * sgm.toFun g2 * sgm.toFun g3 =
    S.inl (sectionCocycle S sgm g1 g2) *
      sgm.toFun (g1 * g2) * sgm.toFun g3

theorem section_triple_product' (sgm : S.Section)
    (g1 g2 g3 : G) :
    sgm.toFun g1 * sgm.toFun g2 * sgm.toFun g3 =
    S.inl (sectionCocycle S sgm g1 g2) *
    S.inl (sectionCocycle S sgm (g1 * g2) g3) *
      sgm.toFun (g1 * g2 * g3)
```

Theorem 5.6 (Cocycle of a splitting)**.** *A splitting has trivial cocycle:*

```
theorem sectionCocycle_of_splitting
    (s : S.Splitting) (g1 g2 : G) :
    sectionCocycle S s.toSection g1 g2 = 1
```

This is fully proved (zero `sorry`) and provides an independent verification of the forward direction of `splits_iff_trivial_cocycle`.

6 Restriction to Subgroups and Local-to-Global

6.1 The direct product extension

As a base case, we formalize the trivially-splitting extension:

Definition 6.1 (Direct product extension)**.**

```
def directProductExtension :
    GroupExtension N (N * G) G where
  inl := { toFun := fun n => (n, 1), ... }
  rightHom := { toFun := fun p => p.2, ... }
  ...
```

Theorem 6.2 (Direct product splits)**.** *The direct product extension splits, with trivial cocycle:*

```
theorem directProduct_splits :
    Nonempty (directProductExtension N G).Splitting

theorem directProduct_has_trivial_cocycle :
    exists sgm :
      (directProductExtension N G).Section,
      forall g1 g2 : G,
        sectionCocycle
          (directProductExtension N G) sgm g1 g2 = 1
```

6.2 The obstruction theorem

The contrapositive of the splitting criterion gives a non-splitting obstruction:

Theorem 6.3 (Non-splitting obstruction)**.** *If every section has a nontrivial cocycle, the extension does not split:*

```
theorem not_splits_of_all_cocycles_nontrivial
    (h : forall sgm : S.Section,
      exists g1 g2 : G,
        sectionCocycle S sgm g1 g2 ≠ 1) :
    ¬Nonempty S.Splitting
```

Proof. Assume for contradiction that a splitting s exists. Then s.`toSection` is a section with trivial cocycle (by `map_mul'` and injectivity of ι), contradicting the hypothesis that every section has a nontrivial cocycle at some pair (g_1, g_2). □

6.3 Restriction via pullback subgroups

Given a subgroup $H \leq G$, the *pullback subgroup* $\pi^{-1}(H) \leq E$ restricts the extension to H:

Definition 6.4 (Pullback subgroup)**.**
```
def pullbackSubgroup (H : Subgroup G) :
  Subgroup E where
  carrier := { e | S.rightHom e ∈ H }
  one_mem' := by simp [H.one_mem]
  mul_mem' := ...
  inv_mem' := ...
```

Definition 6.5 (Restricted extension)**.**
```
def restrictExtension (H : Subgroup G) :
  GroupExtension N (pullbackSubgroup S H) H
```

The restricted extension has:

- $\iota' : N \to \pi^{-1}(H)$ given by $n \mapsto \langle \iota(n), \text{proof that } \pi(\iota(n)) = 1 \in H \rangle$.
- $\pi' : \pi^{-1}(H) \to H$ given by $\langle e, h_e \rangle \mapsto \langle \pi(e), h_e \rangle$.
- Injectivity of ι' from injectivity of ι.
- The range-kernel equation from the original equation restricted to $\pi^{-1}(H)$.
- Surjectivity of π' from surjectivity of π (for each $\langle g, h_g \rangle : H$, choose e with $\pi(e) = g$; then $e \in \pi^{-1}(H)$).

6.4 Global-to-local and the contrapositive

Theorem 6.6 (Global splitting restricts)**.** *A splitting of S restricts to a splitting of $S|_H$ for any subgroup $H \leq G$:*

```
theorem splitting_restricts (H : Subgroup G)
    (s : S.Splitting) :
    Nonempty (restrictExtension S H).Splitting

theorem global_split_implies_local_split
    (H : Subgroup G) :
    Nonempty S.Splitting ->
      Nonempty (restrictExtension S H).Splitting
```

Proof. Given a splitting $s : G \to^* E$, define $s|_H : H \to \pi^{-1}(H)$ by $h \mapsto \langle s(h), \text{proof} \rangle$. The homomorphism property follows from s.`map_one'` and s.`map_mul'`; the section property from s.`rightHom_splitting`. □

Theorem 6.7 (Local obstruction)**.** *If the restricted extension over some subgroup H does not split, then the original extension does not split:*

```
theorem local_nonsplit_obstructs_global
    (H : Subgroup G)
    (h : ¬Nonempty
      (restrictExtension S H).Splitting) :
    ¬Nonempty S.Splitting
```

This is the contrapositive of Theorem 6.6. It provides a *local-to-global obstruction principle*: to show that an extension does not split, it suffices to find a single subgroup over which the restriction does not split. This is a standard technique in group cohomology (restriction maps $H^2(G, N) \to H^2(H, N)$), here formalized at the extension level.

7 Embedding Problems as Extension Splitting

An *embedding problem* in Galois theory asks: given a Galois extension L/K with group G and a surjection $\varphi : H \twoheadrightarrow G$, does there exist a Galois extension F/K with group H extending L/K? The kernel $N = \ker(\varphi)$ gives a group extension $1 \to N \to H \to G \to 1$, and a solution is a homomorphic section $\psi : G \to H$ with $\varphi \circ \psi = \mathrm{id}$—exactly a splitting of the extension.

We formalize this connection at the abstract group-theoretic level (not yet connected to actual Galois extensions via Mathlib's `IntermediateField`).

Definition 7.1 (Embedding problem)**.** An `EmbeddingProblem G H` consists of a surjective group homomorphism $\varphi : H \to G$. The associated group extension is $1 \to \ker(\varphi) \to H \xrightarrow{\varphi} G \to 1$, constructed as `ep.toGroupExtension`.

Theorem 7.2 (Solvability as splitting)**.** *An embedding problem is solvable if and only if the associated group extension splits:*

```
theorem solvable_iff_splits (ep : EmbeddingProblem G H) :
    Nonempty ep.Solution <-> Nonempty ep.toGroupExtension.Splitting
```

Theorem 7.3 (Solvability as trivial cocycle)**.** *Combining with the splitting criterion, an embedding problem is solvable if and only if the associated extension has a section with trivial cocycle:*

```
theorem solvable_iff_trivial_cocycle (ep : EmbeddingProblem G H) :
    Nonempty ep.Solution <->
    exists s : ep.toGroupExtension.Section,
      forall g1 g2 : G,
        sectionCocycle ep.toGroupExtension s g1 g2 = 1
```

This connects the Galois-theoretic lifting question directly to the cohomological obstruction from group extension theory. The architecture decomposes the problem into layers: the bare layer (known Galois group G), the enriched layer (target group H), the fiber (kernel N), and the obstruction (cocycle class). To our knowledge, this is the first Lean formalization connecting embedding problems to group extension splitting with cocycle obstructions.

Scope. The formalization works at the abstract group-theoretic level. The bridge to actual Galois extensions—connecting `EmbeddingProblem` to Mathlib's `IntermediateField` and `IsGalois`—is future work.

8 Faithfully Flat Descent as Conservative Instance

To illustrate the range of the fiber/section/obstruction pattern, we briefly present a contrasting application: faithfully flat descent for ring homomorphisms. This section packages existing Mathlib results (due to others) and is not a contribution in its own right; it serves only to clarify the architectural distinction between conservative and non-conservative forgetful maps.

8.1 The descent principle

For a faithfully flat algebra map $R \to S$, the base-change functor on ring homomorphisms *reflects* structural properties: if the base-changed map $S \otimes_R f$ is injective (resp. surjective, bijective), then f itself is injective (resp. surjective, bijective). Mathlib formalizes this via `RingHom.CodescendsAlong`.

Theorem 8.1 (Descent of injectivity, surjectivity, bijectivity)**.**

```
    theorem faithfullyFlat_reflects_injective :
    RingHom.CodescendsAlong
      (fun f => Function.Injective f)
      RingHom.FaithfullyFlat

theorem faithfullyFlat_reflects_surjective :
    RingHom.CodescendsAlong
      (fun f => Function.Surjective f)
      RingHom.FaithfullyFlat

theorem faithfullyFlat_reflects_bijective :
    RingHom.CodescendsAlong
      (fun f => Function.Bijective f)
      RingHom.FaithfullyFlat
```

These are direct wrappers around Mathlib's `FaithfullyFlat.codescendsAlong_injective` etc., packaged as architecture statements.

8.2 Architectural contrast

The two application domains illustrate complementary instances of the fiber architecture:

Aspect	**Group extensions**	**Faithfully flat descent**
Forgetful map	$\pi : E \to G$ (quotient)	Base change $R \to S$
Surjectivity	Always	Always (for properties)
Section (set-theoretic)	Always exists	N/A (property reflection)
Section (algebraic)	May not exist	Always (conservative)
Obstruction	2-cocycle in N	None (descent is free)

Group extensions exhibit a *non-conservative* forgetful map: the set-theoretic section always exists, but the algebraic section (splitting) requires additional structure (trivial cocycle). Faithfully flat descent exhibits a *conservative* forgetful map: bare properties fully determine enriched properties. The architecture accommodates both cases, with the cocycle measuring the gap in the non-conservative case and vanishing in the conservative case.

9 Related Work

Mathlib's group extension module. Yamazaki's formalization [3] provides the foundational API on which our work builds. The explicit TODOs in `GroupExtension/Defs.lean` (lines 43–52) identify the H^1 and H^2 classifications as targets; our section cocycle and splitting criterion constitute the foundational layer of the H^2 direction.

Group cohomology in Lean. Kull's formalization of low-degree group cohomology [4] provides `IsMulCocycle₂` and `IsMulCoboundary₂`. Livingston's work on the cohomological infrastructure in Mathlib [5] provides the broader framework for group cohomology, including the cochain complex and the definition of H^n. Our 2-cocycle identity (Theorem 5.5) connects the extension-theoretic cocycle to the low-degree predicates; the full bridge—showing that the map [extensions] $\to H^2(G, N)$ is a bijection—remains future work and will require both contributions.

Gonthier et al.'s Feit–Thompson formalization. The Feit–Thompson theorem [6] in Coq/SSReflect is the largest completed formalization in finite group theory. While it does not specifically address the cohomological classification of extensions, its treatment of group actions, transfer, and the Schur–Zassenhaus theorem provides context for our work. Our formalization is considerably smaller in scope but addresses a specific gap in Lean/Mathlib's coverage.

HoTT fiber types. In homotopy type theory [7], fibers of maps are a primitive concept, and the fiber sequence of a map $f : A \to B$ organizes the homotopy-theoretic content. Our `ExtensionFiber` is the set-level analogue: the preimage fiber of a group homomorphism. The torsor structure (Theorem 3.7) is the group-theoretic specialization of the general principle that fibers of a principal bundle are torsors for the structure group.

Mathlib's ShortComplex. Mathlib's homological algebra library includes `ShortComplex` for short exact sequences in abelian categories. Our `GroupExtension` works at the concrete group level rather than the categorical level; connecting the two (showing that a `GroupExtension` gives rise to a `ShortComplex` in the category of groups) is a natural direction for future work.

Other formalizations. To our knowledge, no prior formalization in Lean, Coq, Isabelle/HOL, or Agda provides the section cocycle and splitting criterion at the level of generality presented here, connected to a library-standard cohomological predicate. The standard mathematical references for the underlying theory are Brown [8], Weibel [9], Robinson [10], and Rotman [11]; our contribution is the formalization and its architectural organization, not the mathematics itself.

10 Conclusion

What is proved. We have presented a machine-checked formalization of the section 2-cocycle, splitting criterion, embedding-problem equivalence, and local-to-global obstruction for group extensions in Lean 4. The central theorem package consists of: the splitting criterion `splits_iff_trivial_cocycle` (an extension splits iff some section has trivial cocycle); the embedding-problem equivalence `solvable_iff_trivial_cocycle` (an embedding problem is solvable iff the associated extension splits iff the cocycle vanishes); the 2-cocycle identity `sectionCocycle_isMulCocycle₂_conj` (connecting to Mathlib's low-degree cohomology predicates); and the local-to-global obstruction principle `local_nonsplit_obstructs_global`. All results are fully machine-checked with zero `sorry`.

What this means methodologically. The fiber architecture turns the gap between set-theoretic existence and algebraic structure into an explicit intermediate invariant: the 2-cocycle. This invariant is not merely a classification tool; it is the precise obstruction whose vanishing is equivalent to splitting, and whose connection to embedding problems bridges group extension theory to inverse Galois theory. The combination with faithfully flat descent (conservative forgetful map, no

obstruction) demonstrates that the architecture accommodates a range of algebraic phenomena, from non-conservative to conservative forgetful maps.

Future work. The path toward Mathlib's documented H^2 TODO requires: (1) the H^2 bridge (section cocycle determines a well-defined class in $H^2(G, N)$ depending only on the equivalence class of the extension); (2) the inverse construction (twisted product $N \times_c G$ from a 2-cocycle); (3) the H^1 classification (conjugacy classes of splittings correspond to $H^1(G, N)$). Each step builds on the infrastructure established here. The bridge from abstract embedding problems to actual Galois extensions via Mathlib's `IntermediateField` is a separate direction. Companion work [12] applies the same fiber architecture to Quillen's Theorem A for Galois connections, demonstrating that the pattern reaches beyond group theory into combinatorial topology.

A Theorem Index

The following table maps paper theorems to their Lean names and source files. All results are fully machine-checked with zero `sorry`.

Ref.	Lean name	Source file
Def. 3.1	`ExtensionFiber`	`FiberArchitecture.lean`
Thm. 3.2	`extension_fiber_nonempty`	`FiberArchitecture.lean`
Thm. 3.3	`rightHom_surjective_hasRightInverse`	`FiberArchitecture.lean`
Def. 3.4	`kernelActOnFiber`	`FiberArchitecture.lean`
Thm. 3.5	`kernelAct_free`	`FiberArchitecture.lean`
Thm. 3.6	`kernelAct_transitive`	`FiberArchitecture.lean`
Thm. 3.8	`section_exists`	`FiberArchitecture.lean`
Thm. 3.9	`splitting_iff_section_is_homomorphism`	`FiberArchitecture.lean`
Def. 4.1	`sectionCocycle`	`SchurZassenhaus.lean`
Thm. 4.2	`sectionCocycle_spec`	`SchurZassenhaus.lean`
Thm. 4.3	`section_mul_eq`	`SchurZassenhaus.lean`
Thm. 4.4	`splits_iff_trivial_cocycle`	`SchurZassenhaus.lean`
Def. 4.5	`sectionDiff`	`CocycleCoboundary.lean`
Thm. 4.6	`section_eq_inl_diff_mul`	`CocycleCoboundary.lean`
Thm. 4.7	`splitting_independent_of_section`	`CocycleCoboundary.lean`
Thm. 4.8	`splits_of_any_trivial_cocycle`	`CocycleCoboundary.lean`
Def. 5.1	`sectionConjAct`	`ConjugationAction.lean`
Thm. 5.2	`inl_sectionConjAct`	`ConjugationAction.lean`
Thm. 5.3	`conjAct_inl`	`ConjugationAction.lean`
Thm. 5.4	`sectionConjAct_independent`	`ConjugationAction.lean`
Thm. 5.5	`sectionCocycle_isMulCocycle₂_conj`	`CocycleIdentity.lean`
Thm. 5.6	`sectionCocycle_of_splitting`	`CocycleIdentity.lean`
Def. 6.1	`directProductExtension`	`LocalGlobal.lean`
Thm. 6.2	`directProduct_splits`, `directProduct_has_trivial_cocycle`	`LocalGlobal.lean`
Thm. 6.3	`not_splits_of_all_cocycles_nontrivial`	`LocalGlobal.lean`
Def. 6.4	`pullbackSubgroup`	`LocalGlobal.lean`

continued on next page

(continued)

Ref.	Lean name	Source file
Def. 6.5	`restrictExtension`	`LocalGlobal.lean`
Thm. 6.6	`splitting_restricts`, `global_split_implies_local_split`	`LocalGlobal.lean`
Thm. 6.7	`local_nonsplit_obstructs_global`	`LocalGlobal.lean`
Thm. 8.1	`faithfullyFlat_reflects_injective`, etc.	`FaithfullyFlatDescent.lean`
Def. 7.1	`EmbeddingProblem`, `toGroupExtension`	`EmbeddingProblem.lean`
Thm. 7.2	`solvable_iff_splits`	`EmbeddingProblem.lean`
Thm. 7.3	`solvable_iff_trivial_cocycle`	`EmbeddingProblem.lean`

All source files are located under `InfinityCompression/GeneralMethod/GroupExtension/` (group extension modules), `InfinityCompression/GeneralMethod/Galois/` (embedding problems), and `InfinityCompression/GeneralMethod/Descent/` (faithfully flat descent).

B Module Dependency Graph

The six core group extension modules (presented in this paper) form a layered dependency structure:

```
FiberArchitecture.lean ⟶ SchurZassenhaus.lean
FiberArchitecture.lean ⟶ ConjugationAction.lean
SchurZassenhaus.lean   ⟶ CocycleCoboundary.lean
SchurZassenhaus.lean   ⟶ CocycleIdentity.lean
ConjugationAction.lean ⟶ CocycleIdentity.lean
FiberArchitecture.lean ⟶ LocalGlobal.lean
SchurZassenhaus.lean   ⟶ LocalGlobal.lean
```

`FiberArchitecture.lean` is the root; it depends only on `Mathlib.GroupTheory.GroupExtension.Basic` and `Mathlib.Logic.Function.Basic`. Three additional modules not discussed in this paper (`NonAbelianObstruction.lean`, `SchurMultiplier.lean`, `MathlibAdapter.lean`) explore further directions and adapter patterns. The descent module `FaithfullyFlatDescent.lean` is independent, depending only on `Mathlib.RingTheory.Flat.FaithfullyFlat.Descent`.

References

[1] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction — CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[2] The Mathlib community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pages 367–381, 2020. DOI: 10.1145/3372885.3373824. Mathlib 4 repository: https://github.com/leanprover-community/mathlib4.

[3] Y. Yamazaki. Group extensions in Mathlib. Contribution to Mathlib 4, 2024. https://leanprover-community.github.io/mathlib4_docs/Mathlib/GroupTheory/GroupExtension/Defs.html.

[4] S. Kull. Low-degree group cohomology in Mathlib. Contribution to Mathlib 4, 2024. https://leanprover-community.github.io/mathlib4_docs/Mathlib/RepresentationTheory/GroupCohomology/LowDegree.html.

[5] A. Livingston. Group cohomology in Mathlib. Contribution to Mathlib 4, 2022–2024. https://leanprover-community.github.io/mathlib4_docs/Mathlib/RepresentationTheory/GroupCohomology/Basic.html.

[6] G. Gonthier, A. Asperti, J. Avigad, Y. Bertot, C. Cohen, F. Garillot, S. Le Roux, A. Mahboubi, R. O'Connor, S. Ould Biha, I. Pasca, L. Rideau, A. Solovyev, E. Tassi, and L. Théry. A machine-checked proof of the odd order theorem. In *Interactive Theorem Proving (ITP 2013)*, volume 7998 of *Lecture Notes in Computer Science*, pages 163–179. Springer, 2013.

[7] The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, 2013. https://homotopytypetheory.org/book/.

[8] K. S. Brown. *Cohomology of Groups*, volume 87 of *Graduate Texts in Mathematics*. Springer, 1982.

[9] C. A. Weibel. *An Introduction to Homological Algebra*, volume 38 of *Cambridge Studies in Advanced Mathematics*. Cambridge University Press, 1994.

[10] D. J. S. Robinson. *A Course in the Theory of Groups*, volume 80 of *Graduate Texts in Mathematics*. Springer, 2nd edition, 1996.

[11] J. J. Rotman. *An Introduction to Homological Algebra*. Springer, 2nd edition, 2009.

[12] N. Spivack. *Quillen's Theorem A for Galois Connections: A Machine-Checked Formalization in Lean 4*. Companion manuscript, 2026.